

CS 21 Refined Design: Card Gala

Abdullahi Abdi, Kevin Arellano Flores, Ethan Goldman, Alex Lee

Project Description Minimum Deliverable (Design Proposal):

Our goal was to have people be able to join a centralized lobby from multiple devices from which they will be able to choose from a list of games to join/create a room that is an instance of that game. Each game will have a cap for the number of players that can join. Once that cap has been reached the game will begin and other players will be prevented from joining that instance of the game and will have to create a new instance if they want to play that type of game. There will be a limit to the number of instances of each type of game that can be created. At least one of the games that users can choose from will be played concurrently, others may be sequential. Examples of concurrent games we can implement are Press the Button and Egyptian Ratscrew. Examples of sequential games include Solitaire and Blackjack. We wanted to host the main lobby on the Tufts Cluster or some other constantly running machine so players can connect to the server at any time from any location and play until they choose to leave the main lobby. **Stretch goal:** Have the users interact with the main lobby and instances of individual games using a custom built GUI.

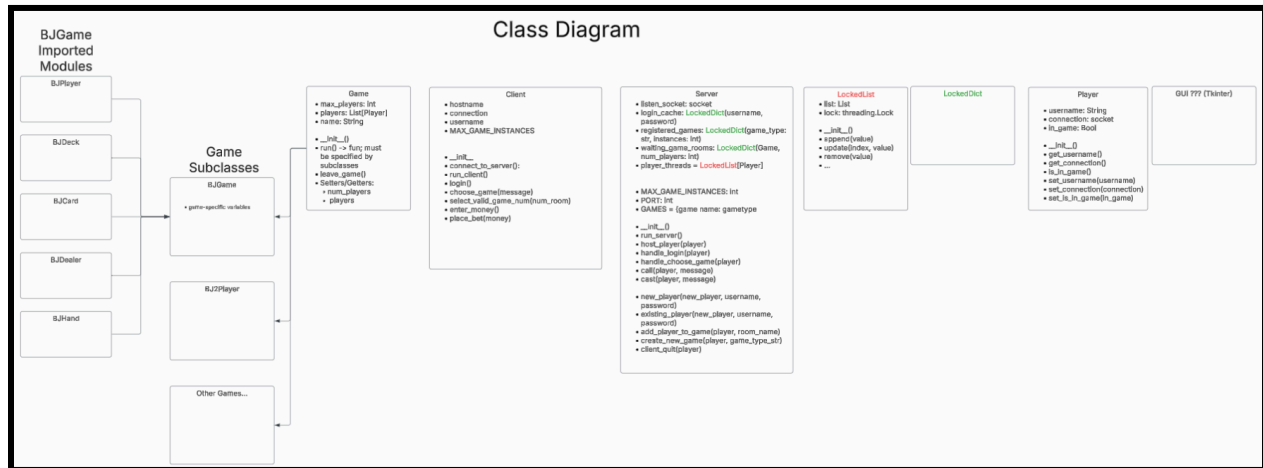
The two main strategies that we considered for developing this game were using a Erlang gen-server to build a distributed type system with Erlang Ports linked to other Python scripts that would control the individual game-play OR to build a system entirely from Python using Python TCP-sockets to host the main lobby, and threading to control the individual games that are created from the main lobby. (We were originally planning on using multiprocessing to run the individual games but complications with maintaining threaded locks and with multiprocessing on windows machines meant that we opted to do everything with threading.) We decided to use the 100%-Python design as we are more familiar with that language and felt that it was easier to design and build everything using a single language.

The way in which a user would connect to the Cardgala server was different for each of the two aforementioned strategies as well. For the Erlang design, users would have to ssh into the Tufts CS Department's Halligan Server from which messages would be broadcasted to other processes running on the Halligan Server (including the Cardgala server) which would mean that users could only play Cardgala if they were on the Halligan Server. With the Python TCP-socket strategy, users could connect to the server directly via public IP which means that users only need access to the network to play anywhere world-wide.

There were also Erlang vs. Python considerations for the GUI but seeing as the Python GUI capabilities are more powerful and since we were using Python for the rest of the project we choose to use Python.

The solution that we came up with for this problem is as follows:

UML Diagram:



We have 3 main classes.

1. A **Server** class that is in charge of instantiating/managing Game processes and brokering data between running Games and clients/players. The server class also runs a producer-consumer model that handles incoming connections from clients by creating player objects for those clients and initiating the “login” and “choose game” sequences for those player objects which run on their own thread. The server class maintains the game state for all players, games, etc.
2. A **Client** class that handles client I/O and stores client specific information such as the client-specific TCP-socket connection between the client and the server.
3. A **Player** class that is used in the server to store client-related data.
4. A **Game** superclass that contains data needed to run/manage games in general (maximum number of players, the name of the game, the list of players in the game, etc.). The game class contains a run() function that must be instantiated by subclasses. The run() function is the game-specific function that defines the game flow for each game.
 - a. **BJGame, BJTwoPlayer, PTBGame, and CrazyEight** all of which are game subclasses. They define the four different games that a player may play on cardgala. Each game subclass imports other submodules.
5. **LockedList** and **LockedDict** classes are used to protect mutable shared lists/dictionaries

Following the basic, functional implementation of Cardgala, we began work on the development of a GUI. We sought to provide users with the capacity to intuitively navigate Cardgala without the need for background context. The development of the GUI was dependent on our understanding of the various cases/edge-cases present in each game, and in the Cardgala architecture. We manually created ASCII art via ASCII Draw Studio to immerse the user in an environment reminiscent of a casino, for example, by making visuals for the different cards in a deck used across games; This art was saved in a dedicated file for convenient access, where a

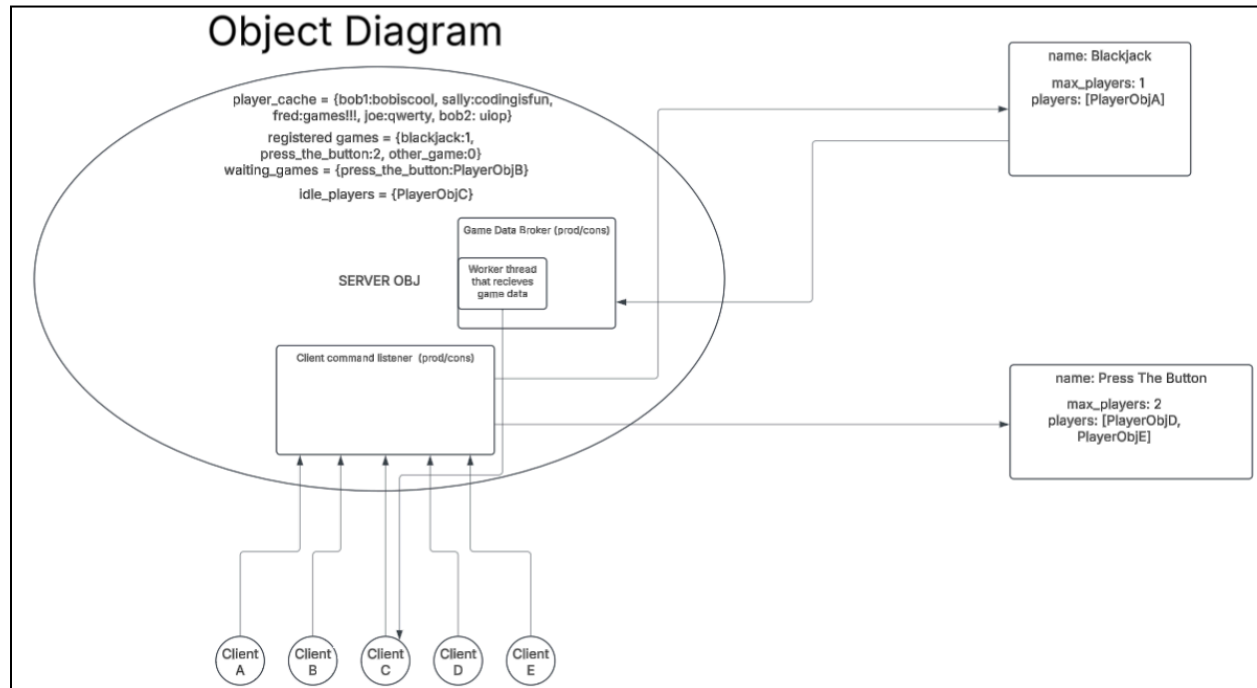
dictionary maps the name of objects to their corresponding visual representation. Additionally, we used the methods of Python's colorama library to add a variety in the color, opacity, and font of text displayed on the user's terminal. We leveraged connotations commonly attributed to different colors to create contrast across the vast amounts of text that a user may see at-once, for example: using red to indicate the occurrence of errors, green to indicate successful operations, etc.

Abstraction/Language:

One major abstraction is the information that the server object has vs. the information that the client objects have. The server object knows the game state of every game, which users are connected to the server, their states, etc. while the client objects that are communicating with the server over a network at a given time only know data relevant to the current game state that they are in or the relevant lobby information given to them at the discretion of the server. This abstraction greatly simplifies the client code as the client is only performing basic user I/O. It also makes the GUI design easier as the server doesn't have to maintain the GUI and then broadcast the GUI to the clients, instead each client knows what to display to the GUI based on which game state they are currently in, what user input needs to take place, etc.

A new form of "language" we have introduced to our project comes in the form of commands given to our modules to modify either the server, client, or game state, depending on the context. The aforementioned commands are saved as a suite of python dictionaries, which we refer to in code as "states". These states mirror the formatting of typical client/server casts and calls, containing message keywords (names) and any relevant data needed along with the message. Certain modules are set up to receive certain states. The Blackjack game module, for example, will only respond to a message or modify its game state when it receives state messages that are Blackjack specific – other modules (namely the server and client) follow essentially this same protocol. Allowing our modules to pass specific messages amongst each other in this way allows us to better organize interactions between the continuously interleaving states of our program. We opted to store these messages in a file accessible to all modules called `states.py`, which provides us with a "menu" of commands that certain objects can send and receive, effectively providing an additional level of cleanliness from a programmer's perspective.

Object Diagram:



Two clients have joined the server. All five have created a username and password which are now stored in the `player_cache`. There are three types of games that players can play - blackjack, `push_the_button` and `other_games`. There is currently one process running an instance of the blackjack game which has been created by player A and has started to run because only one player is required to play blackjack. There is one process running an instance of `push_the_button` being played by Players D and E. In order for that game to begin, one of those two players had to create the game and the other had to join it because two players are required to play `push_the_button`. There is one instance of the `push_the_button` game that is waiting to begin - Player B has created the game but no one else has joined so the required number of players has not yet been reached, hence why there is no process running a second instance of this game (the fact that the game has been created but is not yet being played is stored in `waiting_games`). There is currently one idle player sitting in the main server, Player C, who can choose to join the waiting `push_the_button` game or who can choose to create their own instance of one of the three games. There are a few producer/consumer models living in the server that are handling message passing between the server, clients, and game processes.

Analysis of the outcome:

The minimum deliverable was to have people be able to join a centralized lobby from multiple devices from which they will be able to choose from a list of games to join/create a room that is an instance of that game. At least one of the games that users can choose from will be played concurrently, others may be sequential. We were able to achieve the minimum deliverable, because we were able to create a centralized lobby for users to choose a game to create. As well as having at least one of the games be concurrently pushed the button. Our maximum deliverable was to have 4 games and we were able to complete that. What went right in our project to get us to be able to achieve maximum deliverable was being able to get the server and lobby up and running in the first few weeks giving us time to focus on creating the games.

Reflection on our design:

Our design was particularly effective at maintaining modularity between the implementation of server/client protocols of the lobby and game-specific logic while maintaining an easy way to communicate between the two modules. This was accomplished using our choice of message passing with our “states” protocol. This decision allowed our game modules to communicate with the server and, by proxy, the client, without needing to know the specific technical implementations of the connection, which made our game creation process easily scalable. A design choice that was potentially a drawback was the decision to hard-code state messages for each module that needed to communicate with the server. While this allowed for limiting each server-module connection to a specific set of commands and subsequent logic, it meant that we would need to re-implement certain commands that were needed across multiple different connection states. For example, many of these states implement the same “printing” command, which simply prints a message to the client from the calling module. While we could have opted to make a suite of “global” states to spare code reuse, due to the way we associated each module with its specified state commands mapping, it would require us to deviate from the design choice we made. In future iterations of Card-GALA, one may consider re-working the state mapping system slightly to minimize code reuse between modules.

Reflection on the division of labor:

The work was divided such that one team member designed the server/client interface, one team member designed the GUI, and the other two team members designed the games. Once all individual components were roughly complete, we integrated them together. This integration

necessitated that all team members learn how the other parts of the code worked which was actually quite useful because it helped us to identify bugs in the individual components and helped us to clean up certain sections of the code that were difficult to read, producing cleaner and better code.

The way in which our team divided the project allowed us to work efficiently as a team as we were able to divide and conquer but then combine our individual components efficiently. We also did a good job budgeting enough time for the integration so that we never were really in a time crunch throughout the entire project build.

Bug report:

This may not have been the most difficult bug to find but the most exciting bug to resolve was related to how we manage the games while they are running. We were originally planning on running each game on its own process using the python multiprocessing library. However, while we were able to set this up and got it to run correctly for games where all the clients were connected over localhost, we had issues playing games on machines that ran windows and we had a really large issue created by the fact that you can't pass threading locks into new processes. We decided therefore to run individual games on threads instead of on processes so we could pass threading locks to the game threads which was really important for maintaining certain game states such as the lists of players in specific games. Figuring out what the issue was and deciding to attempt to use threading instead took about 30 minutes and it was really exciting to be able to remote into the server on all of our machines and play together for the first time. In retrospect, we probably should have tested out playing on the server across different machines sooner (we had been doing most of our testing on localhost up to that point) which would have made debugging that issue less stressful.